

EGERTON UNIVERSITY



**FACULTY OF SCIENCE
DEPARTMENT OF COMPUTER SCIENCE**

COMP 493: COMPUTER SYSTEMS PROJECT

DESIGN AND IMPLEMENTATION OF A STUDENT ATTACHMENT MANAGEMENT SYSTEM (SAMS)

Submitted by:

Melanie Asiko

REG. NO: S13/07748/22

Supervisor:

Mr Njoroge

Coordinator:

Mr Bosire

Submitted in Partial Fulfillment of the Requirements for the Award of
Bachelor of Science in Computer Science

May, 2026.

DECLARATION

I, Melanie Asiko, hereby declare that this project titled “*Design and Implementation of a Student Attachment Management System (SAMS)*” is my original work and has not been submitted elsewhere for academic purposes.

All sources used have been acknowledged.

Signature: _____

Date: _____

Supervisor Signature: _____

ACKNOWLEDGEMENT

I thank God for the strength and guidance throughout this project.

I sincerely appreciate my supervisor, Mr Njoroge, for their continuous support and guidance.

I also thank the Department of Computer Science, Egerton University, for providing the necessary resources.

TABLE OF CONTENT

EGERTON UNIVERSITY	0
COMP 493: COMPUTER SYSTEMS PROJECT	0
DESIGN AND IMPLEMENTATION OF A STUDENT ATTACHMENT MANAGEMENT SYSTEM (SAMS)	0
DECLARATION	1
ACKNOWLEDGEMENT	2
TABLE OF CONTENT	3
PROJECT PROPOSAL	5
Title: Design and Implementation of a Student Attachment Management System	5
CHAPTER 1: INTRODUCTION	6
1.1 Literature Review	6
1.2 Problem Statement	6
1.3 Justification	7
1.4 Research Questions	7
1.5 Research Objective	7
CHAPTER 2: PROPOSED SOLUTION	7
2.1 Requirements Analysis	7
Functional Requirements	7
Non-Functional Requirements	8
2.2 Planning	8
2.3 Architectural Design	8
2.4 Software Development	8
2.5 Testing	9
2.6 Deployment	9
CHAPTER 3: FINAL RESULTS	9
CHAPTER 4: CHALLENGES, OBSTACLES, AND RISKS	10
CHAPTER 5: SCHEDULE AND BUDGET	10
5.1 Schedule	10
5.2 Budget	10
CHAPTER 6: CONCLUSIONS AND FUTURE WORKS	11
6.1 References	11
SOFTWARE REQUIREMENTS SPECIFICATION (SRS)	12
1. Introduction	12
1.1 Purpose	12
1.2 Scope	12
2. Overall Description	12
2.1 System Overview	12
2.2 User Roles	12
3. Functional Requirements	13

3.1 Authentication	13
3.2 Student Portal	13
3.3 Coordinator Portal	13
4. System Flowchart	14
System Authentication & Routing Flow	14
Logbook Submission Flow	15
SOFTWARE DESIGN DOCUMENT (SDD)	16
1. System Architecture	16
1.1 Architecture Diagram	16
2. Database Design (ER Diagram)	17
ER Diagram	17
3. Module Design	18
3.1 Authentication Module	18
3.2 Student Module	18
3.3 Logbook Module	18
3.4 Coordinator Module	18
4. Sequence Diagram (Login)	18
5. Data Flow Design	19
Dashboard Data Flow	19
6. UI Design Considerations	19
7. Security Design	19
8. Future Enhancements	19
4. TEST PLAN	20
1.0 Introduction	20
1.1 Test Objectives	20
1.2 Scope of Testing	21
1.2.1 Functionality Testing	21
• Validate that all key modules of SAMS perform as designed.	21
• Test user registration, login, and password reset flows.	21
• Test student record management: create, read, update, and delete (CRUD) operations.	21
• Test attendance marking and retrieval by date, class, and student.	21
• Test report generation and PDF/CSV export functionality.	21
• Test admin dashboard: user management, system configuration, and report generation.	21
1.2.2 Integration Testing	21
• Ensure seamless interaction between modules (API endpoints, database, and frontend).	21
• Verify that API endpoints correctly communicate with the Angular frontend.	21
• Validate that database transactions are accurately reflected in the UI after attendance is recorded.	21
• Confirm that authentication tokens generated on the backend are validated for	

protected frontend routes.	21
1.2.3 Performance Testing	21
• Use Apache JMeter to simulate multiple concurrent users (targeting 40+ users).	21
• Measure response times, throughput, and resource usage.	21
• Verify critical operations (login, attendance submission) complete within acceptable limits.	21
• Assess response times under the constraints of the hosting environment (512 MB RAM, 2 vCPU).	21
1.2.4 Security Testing	21
• Verify JWT-based authentication — protected endpoints must reject requests without valid tokens.	21
• Role-Based Access Control (RBAC): admin-only routes must be inaccessible to standard users.	22
• Input validation: SQL injection and XSS attempts must be rejected by the backend.	22
• Sensitive data (passwords, tokens) must be encrypted at rest and in transit.	22
1.2.5 User Acceptance Testing (UAT)	22
• Representative end users (lecturers, students, administrators) participate in structured UAT sessions.	22
• Participants complete predefined task scenarios and provide feedback using standardised forms.	22
• UAT findings are used to identify usability gaps prior to final deployment.	22
3.0 Test Cases	24
5. USER MANUAL	28
5.1 Getting Started	28
5.2 Student Guide	28
Profile	28
Dashboard	28
Logbook	28
Document Vault	28
5.3 Coordinator Guide	28
Dashboard	28
Students Page	28
Supervisor Assignment	29
Company Data	29
5.4 Navigation	29
5.5 Troubleshooting	29

PROJECT PROPOSAL

Title: Design and Implementation of a Student Attachment Management System

CHAPTER 1: INTRODUCTION

1.1 Literature Review

Student attachment or industrial training programs are an important component of higher education because they allow students to gain practical experience in real working environments. Universities use attachment programs to bridge the gap between theoretical knowledge learned in class and practical industry skills.

Traditional attachment management processes in many universities rely heavily on manual methods such as paper logbooks, physical supervision reports, and manual tracking of students' placements. Research shows that digital systems for managing internships and attachments improve efficiency, transparency, and communication between students, supervisors, and institutions.

A Student Attachment Management System can automate important processes such as placement registration, weekly logbook submission, supervision monitoring, and performance evaluation. According to studies in education management systems, web-based monitoring platforms allow supervisors to track student progress remotely and ensure compliance with institutional requirements.

Therefore, implementing a digital attachment management platform can significantly improve the monitoring and evaluation of students during their industrial training period.

1.2 Problem Statement

Many universities still manage student attachments using manual processes such as paper logbooks, email communication, and physical submission of reports. These manual methods create several challenges including:

- Difficulty tracking student placement locations
 - Delayed supervision and monitoring
 - Loss or damage of physical logbooks
- Inefficient evaluation and grading processes
- Lack of real-time visibility of student progress

These limitations make it difficult for academic supervisors and administrators to effectively monitor student activities and ensure that learning objectives are achieved.

1.3 Justification

The proposed Student Attachment Management System will digitize and streamline the entire attachment process from placement registration to final certification.

The system will allow students to submit weekly reports online, supervisors to monitor progress through dashboards and geospatial maps, and administrators to track placements across departments.

This system will improve efficiency, reduce paperwork, enhance transparency in supervision, and ensure better management of student industrial attachments.

1.4 Research Questions

1. How can a web-based system improve the management of student attachment programs?
2. How can supervisors effectively monitor student progress during industrial attachment?
3. How can digital logbooks improve record keeping and reporting during attachments?

1.5 Research Objective

To design and implement a web-based Student Attachment Management System that improves monitoring, reporting, and evaluation of students during their industrial attachment.

CHAPTER 2: PROPOSED SOLUTION

2.1 Requirements Analysis

Functional Requirements

The system will provide the following features:

- Student registration and authentication
- Placement submission including company details and location
 - Weekly logbook submission with file upload
 - Supervisor monitoring dashboard
 - Geospatial map showing student locations
 - Field visit recording and grading
 - Final report submission
 - Automatic certificate generation

Non-Functional Requirements

- Usability: The system must be easy to use for students and supervisors
 - Security: Only authorized users should access the system
- Reliability: The system should store and retrieve data accurately
- Performance: The system should respond quickly to user requests

2.2 Planning

The project will follow the **Software Development Life Cycle (SDLC)** which includes:

1. Requirements Analysis
2. System Design
3. System Development
4. Testing
5. Deployment
6. Maintenance

This approach ensures the system is developed in a structured and organized manner.

2.3 Architectural Design

The system will use a **client-server architecture** where users interact with the system through a web browser while the application server processes requests and communicates with a centralized database.

The system will consist of three main user portals:

- Student Portal
- Academic Supervisor Portal
- Administrative Portal

All user data and system information will be stored in a central database.

2.4 Software Development

The system will be developed using the following technologies:

Frontend:

- Angular
- HTML
- CSS
- TypeScript

Backend:

- Python(FastAPI)

Database:

- MySQL

Additional tools such as **Google Maps API or Leaflet** may be used to implement the geospatial placement map feature.

2.5 Testing

Testing will be conducted to ensure that the system functions correctly and meets user requirements. The testing process will include:

- Unit Testing: Testing individual modules of the system
- Integration Testing: Ensuring system components work together properly
- System Testing: Testing the complete system functionality

These tests will help identify and fix errors before system deployment.

2.6 Deployment

The system will be deployed on a **web server** either locally within the university network or on a cloud hosting platform.

Authorized users such as students, supervisors, and administrators will access the system through secure login credentials.

CHAPTER 3: FINAL RESULTS

The final system will provide a digital platform for managing student attachment activities. It will allow students to register placements, submit weekly logbooks, and upload final reports online.

Supervisors will be able to monitor student progress, verify submissions, conduct field visit evaluations, and assign grades.

Administrators will have oversight of all students on attachment and access to analytics regarding placement industries and departmental participation.

The system will significantly improve monitoring, efficiency, and transparency in the attachment process.

CHAPTER 4: CHALLENGES, OBSTACLES, AND RISKS

Some potential challenges in the implementation of the system include:

- Limited internet connectivity in remote attachment locations
- Resistance to adopting digital systems from users accustomed to manual processes
 - Data security and privacy concerns
- Technical difficulties during system deployment

These challenges can be addressed through proper training, system security measures, and reliable hosting infrastructure.

CHAPTER 5: SCHEDULE AND BUDGET

5.1 Schedule

The project will be completed within one academic semester. The major phases will include system analysis, design, development, testing, and final deployment according to an agreed project timeline.

5.2 Budget

The estimated budget will mainly cover:

- Web development tools
- Hosting and server services
- Internet connectivity for testing
- Maintenance and system updates

The project will prioritize **open-source technologies** in order to minimize development costs.

CHAPTER 6: CONCLUSIONS AND FUTURE WORKS

The proposed Student Attachment Management System demonstrates how digital platforms can improve the management and monitoring of industrial attachment programs in universities.

By replacing manual processes with automated workflows, the system will enhance efficiency, accuracy, and communication between students, supervisors, and administrators.

6.1 References

Laudon, K. C., & Laudon, J. P. *Management Information Systems*.

Pressman, R. S. *Software Engineering: A Practitioner's Approach*.

Sommerville, I. *Software Engineering*.

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

1. Introduction

1.1 Purpose

This document defines the functional and non-functional requirements of the **Student Attachment Management System (SAMS)**. The system enables students to manage their industrial attachment while allowing coordinators to monitor and supervise progress.

1.2 Scope

SAMS provides:

- Student registration and login
- Attachment tracking (start/end dates, progress)
 - Weekly logbook submission
 - Supervisor assignment
- Coordinator dashboard with analytics

2. Overall Description

2.1 System Overview

The system consists of:

- **Frontend:** Angular
- **Backend:** FastAPI
- **Database:** PostgreSQL

2.2 User Roles

- Student
- Coordinator (Admin)

3. Functional Requirements

3.1 Authentication

- Users log in using email and password
- System determines role automatically

3.2 Student Portal

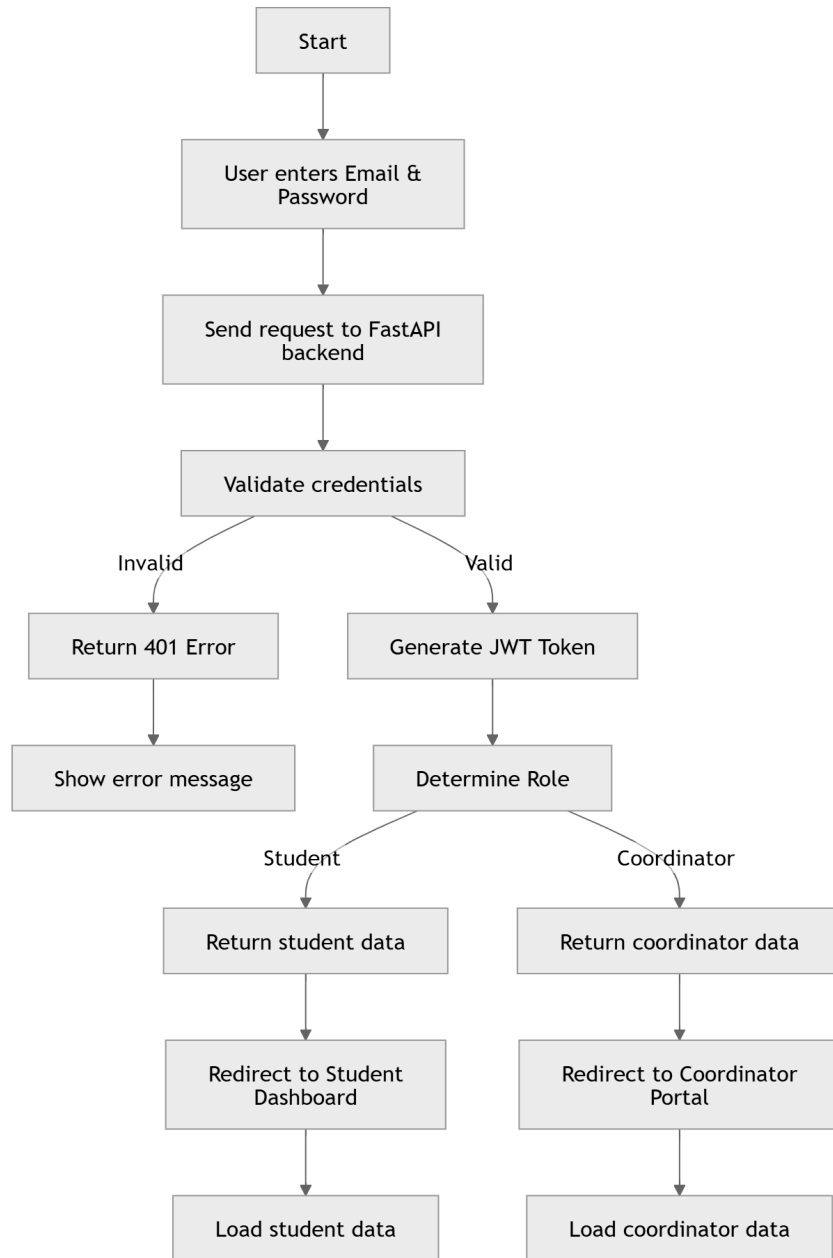
- View dashboard (progress, days remaining)
 - Submit weekly logs
 - Upload documents
 - View supervisors

3.3 Coordinator Portal

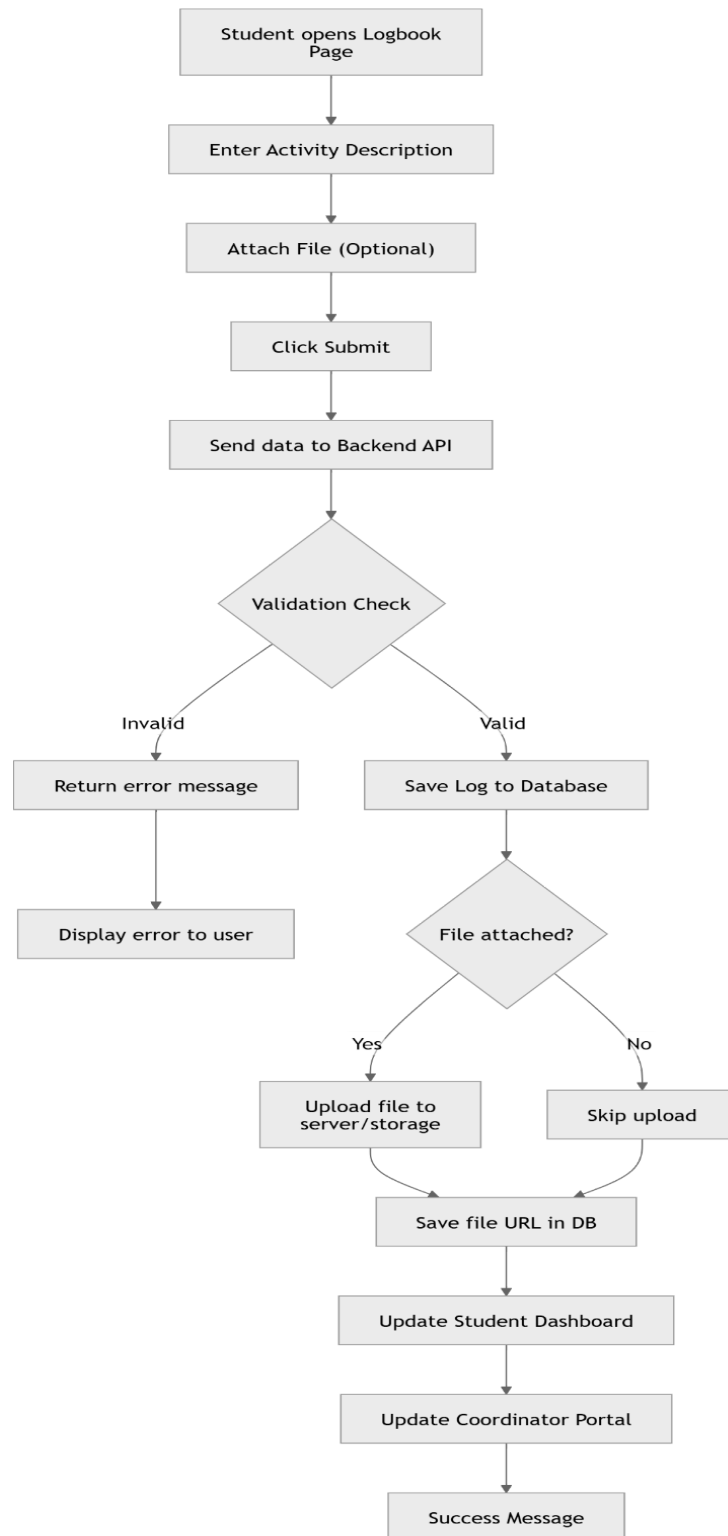
- View all students
- Assign supervisors
- View logs and documents
 - View analytics

4. System Flowchart

System Authentication & Routing Flow



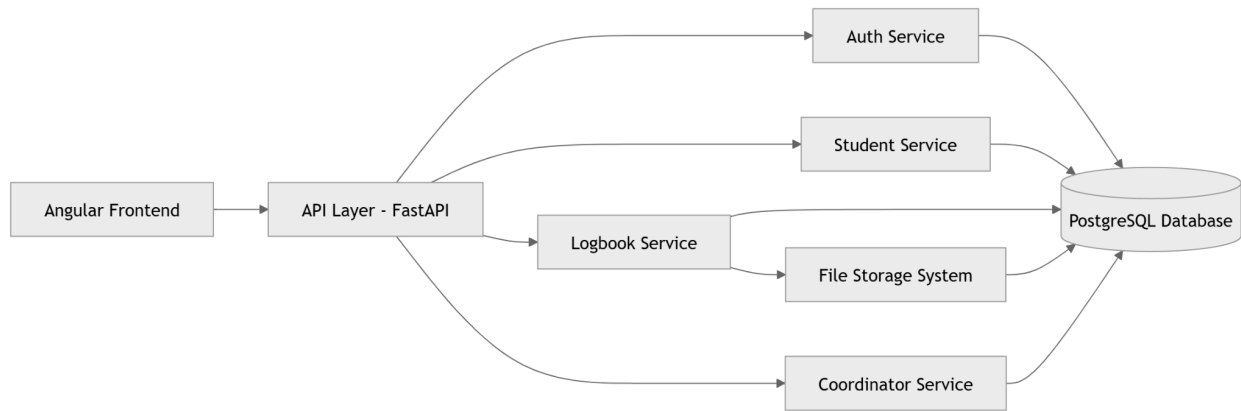
Logbook Submission Flow



SOFTWARE DESIGN DOCUMENT (SDD)

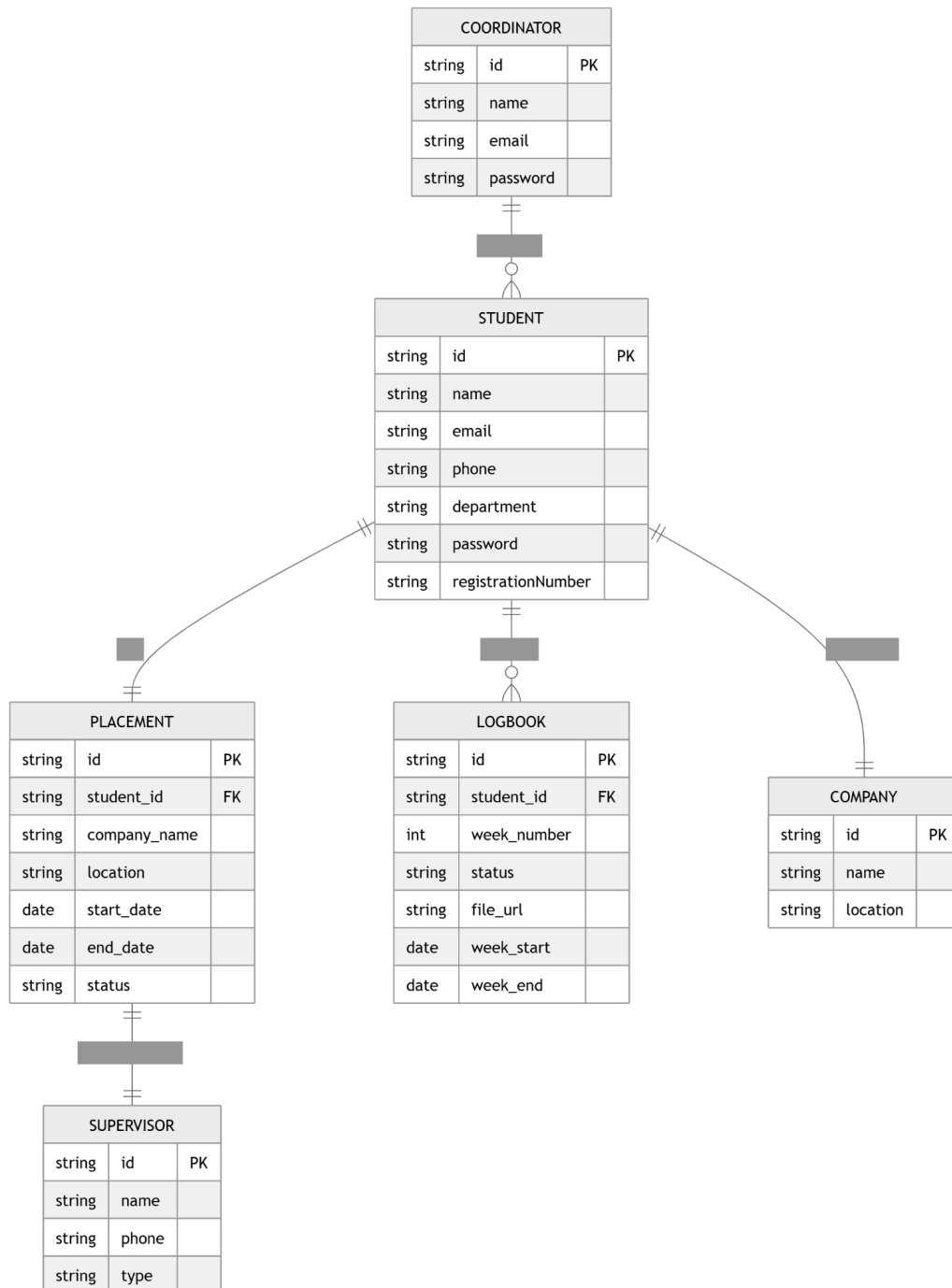
1. System Architecture

1.1 Architecture Diagram



2. Database Design (ER Diagram)

ER Diagram



3. Module Design

3.1 Authentication Module

- Handles login
- Returns token + role
- Controls routing

3.2 Student Module

- Fetch student profile
- Compute progress dynamically
- Manage placement info

3.3 Logbook Module

- Submit logs
- Upload files
- Track weekly submissions

3.4 Coordinator Module

- Manage students
- Assign supervisors
- View reports

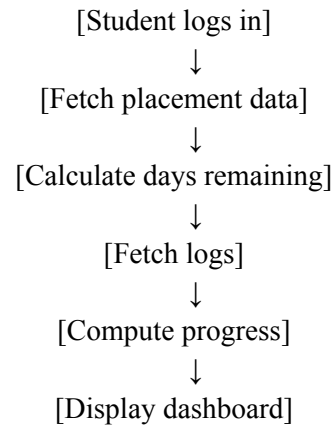
4. Sequence Diagram (Login)

User → Frontend → Backend → Database

User enters credentials
Frontend sends request
Backend validates user
Database returns user
Backend sends token
Frontend redirects user

5. Data Flow Design

Dashboard Data Flow



6. UI Design Considerations

- Clean dashboard layout
 - Responsive design
 - Real-time updates
- Error handling messages

7. Security Design

- JWT authentication
- Password hashing
- Role-based access control

8. Future Enhancements

- Real-time notifications
- Map integration (student locations)
 - Mobile app version

4. TEST PLAN

1.0 Introduction

This Test Plan document outlines the testing strategy, scope, test cases, and reporting procedures for the Student Attendance Management System (SAMS). SAMS is a web-based application developed to automate and streamline the management of student attendance records for academic institutions. The system comprises a Python-based backend, a TypeScript/Angular frontend, and a relational database layer.

The testing process is designed to ensure that all functional and non-functional requirements are met, that the system integrates correctly across its components, and that it delivers a reliable, secure, and user-friendly experience.

1.1 Test Objectives

The main objectives of the testing phase include:

- Validating the functionality of all modules including student records, attendance tracking, and user authentication.
- Verifying system integration between the Angular frontend, Python backend API, and the relational database.
- Ensuring system performance under varying concurrent user loads.
- Checking security mechanisms, including JWT-based authentication and role-based access control (RBAC).
- Assessing that the system is usable and meets end-user needs through User Acceptance Testing (UAT).

1.2 Scope of Testing

1.2.1 Functionality Testing

- Validate that all key modules of SAMS perform as designed.
 - Test user registration, login, and password reset flows.
- Test student record management: create, read, update, and delete (CRUD) operations.
 - Test attendance marking and retrieval by date, class, and student.
 - Test report generation and PDF/CSV export functionality.
- Test admin dashboard: user management, system configuration, and report generation.

1.2.2 Integration Testing

- Ensure seamless interaction between modules (API endpoints, database, and frontend).
 - Verify that API endpoints correctly communicate with the Angular frontend.
- Validate that database transactions are accurately reflected in the UI after attendance is recorded.
- Confirm that authentication tokens generated on the backend are validated for protected frontend routes.

1.2.3 Performance Testing

- Use Apache JMeter to simulate multiple concurrent users (targeting 40+ users).
 - Measure response times, throughput, and resource usage.
- Verify critical operations (login, attendance submission) complete within acceptable limits.
- Assess response times under the constraints of the hosting environment (512 MB RAM, 2 vCPU).

1.2.4 Security Testing

- Verify JWT-based authentication — protected endpoints must reject requests without valid tokens.

- Role-Based Access Control (RBAC): admin-only routes must be inaccessible to standard users.
- Input validation: SQL injection and XSS attempts must be rejected by the backend.
- Sensitive data (passwords, tokens) must be encrypted at rest and in transit.

1.2.5 User Acceptance Testing (UAT)

- Representative end users (lecturers, students, administrators) participate in structured UAT sessions.
- Participants complete predefined task scenarios and provide feedback using standardised forms.
- UAT findings are used to identify usability gaps prior to final deployment.

2.0 System Performance Report

The purpose of this report is to evaluate the performance of SAMS under realistic operating conditions. The frontend is hosted on a static hosting platform and the backend is deployed on a cloud hosting service (Railway). The database currently contains test data representing 40 users and associated attendance records.

2.1 Test Environment and Tools

Environment:

- Frontend: Angular application hosted on a CDN-backed static hosting service.
- Backend: Python API deployed on Railway — 512 MB RAM, 2 vCPU.
- Database: Relational database (MySQL/PostgreSQL) with seed data for 40 users.

Tools:

- Unit Testing: pytest (backend) and Jasmine/Karma (Angular frontend components).
- Integration Testing: Postman for API endpoint validation and contract testing.
- System Testing: Selenium WebDriver for end-to-end browser automation.
- Performance Testing: Apache JMeter for concurrent user load simulation.

- Security Testing: Postman manual testing, OWASP ZAP for vulnerability scanning.

2.2 Performance Testing Objectives

- Measure Response Time: Evaluate how quickly the system processes key operations.
- Assess Throughput: Determine the number of successful transactions per second.
- Simulate Concurrent Users: Test with 40+ simultaneous users.
- Evaluate Resource Utilisation: Monitor CPU and RAM usage on the hosted backend.

2.3 Performance Results

Test Scenario	Avg Response Time (s)	Throughput (req/s)	Error Rate (%)
User Login	2.1	14	0.3
Student Record Fetch	2.8	11	0.5
Attendance Submission	3.0	10	0.7
Report Generation	4.2	8	1.0
PDF Export	5.1	6	1.2

Observations:

- User Login: Response times remained consistently below 3 seconds, meeting the performance target.
- Student Record Fetch: Acceptable retrieval times; minor degradation observed beyond 35 concurrent users.
- Attendance Submission: Processed within acceptable limits with a low error rate.

- Report Generation: Slightly elevated response times due to database aggregation queries.
- Resource Utilisation: Backend operated within 512 MB RAM / 2 vCPU constraints under test loads.

3.0 Test Cases

3.1 TCSA01: User Login with Valid Credentials

Test Case ID: TCSA01

Priority: High

Module: Authentication

Description: Verifies that a registered user can successfully log into SAMS using valid credentials.

Preconditions:

- A user account exists in the database with known credentials.
- The SAMS login page is accessible via a web browser.

Test Steps:

1. Navigate to the SAMS login page.
2. Enter a valid email address (e.g., testuser@egerton.ac.ke).
3. Enter the corresponding valid password (e.g., SecurePass@2024!).
4. Click the "Login" button.
5. Observe the redirect behaviour and session token generation.

Expected Result: The system authenticates the user, generates a valid JWT session token, and redirects to the main dashboard. A personalised welcome message is displayed.

Actual Result: User logs in successfully and is redirected to the dashboard with all navigation elements visible.

Status: Pass

Comments: Ensure that the system handles valid credentials without noticeable delays.

3.2 TCSA02: User Login with Invalid Credentials

Test Case ID: TCSA02

Priority: High

Module: Authentication

Description: Verifies that SAMS rejects login attempts with incorrect credentials.

Preconditions:

- The login page is accessible.
- The test credentials do not correspond to any registered account.

Test Steps:

1. Navigate to the SAMS login page.
2. Enter an invalid email (e.g., notouser@example.com).
3. Enter an incorrect password (e.g., WrongPass999!).
4. Click the "Login" button.
5. Observe the system response.

Expected Result: The system rejects the login, displays an error message (e.g., "Invalid email or password"), and does not generate a session token.

Actual Result: Error message displayed; no token issued; user remains on login page.

Status: Pass

Comments: Confirm that error messages do not expose sensitive system information.

3.3 TCSA04: Password Reset Process

Test Case ID: TCSA04

Priority: Medium

Module: Authentication

Description: Verifies that the password reset flow works correctly.

Test Steps:

1. On the login page, click "Forgot Password?".
2. Enter a registered email address (e.g., jane.doe@egerton.ac.ke).
3. Click "Send Reset Link".
4. Open the email client and click the password reset link.
5. Enter and confirm a new password (e.g., NewSecurePass789!).
6. Click "Reset Password".
7. Attempt to log in using the new password.

Expected Result: A secure reset link is sent. After setting a new password, the user can log in successfully.

Actual Result: Reset link received; new password accepted; login successful.

Status: Pass

5. USER MANUAL

5.1 Getting Started

1. Open the system
2. Register as a student
3. Log in

5.2 Student Guide

Profile

- Update phone, department, supervisor info

Dashboard

- View:
 - Progress
 - Days remaining
 - Placement details

Logbook

1. Enter weekly activity
2. Upload file
3. Submit

Document Vault

- View uploaded files

5.3 Coordinator Guide

Dashboard

- View total students
- Monitor placements

Students Page

- Search students

- View details

Supervisor Assignment

- Assign lecturers to students

Company Data

- Auto-generated from student input

5.4 Navigation

- Login → redirects automatically
 - Menu links:
 - Dashboard
 - Students
 - Lecturers
 - Companies

5.5 Troubleshooting

- Login issues → check credentials
 - Missing data → refresh
- Upload issues → check file size